Summarize All The Things

Don Syme (dsyme@github.com) + AI4PRs Squad + all of GitHub Next

Summarization: Rationale

Soon **most text associated with code** will be **emergent** and **automatic.**

Content ⇒ Text

Variations:

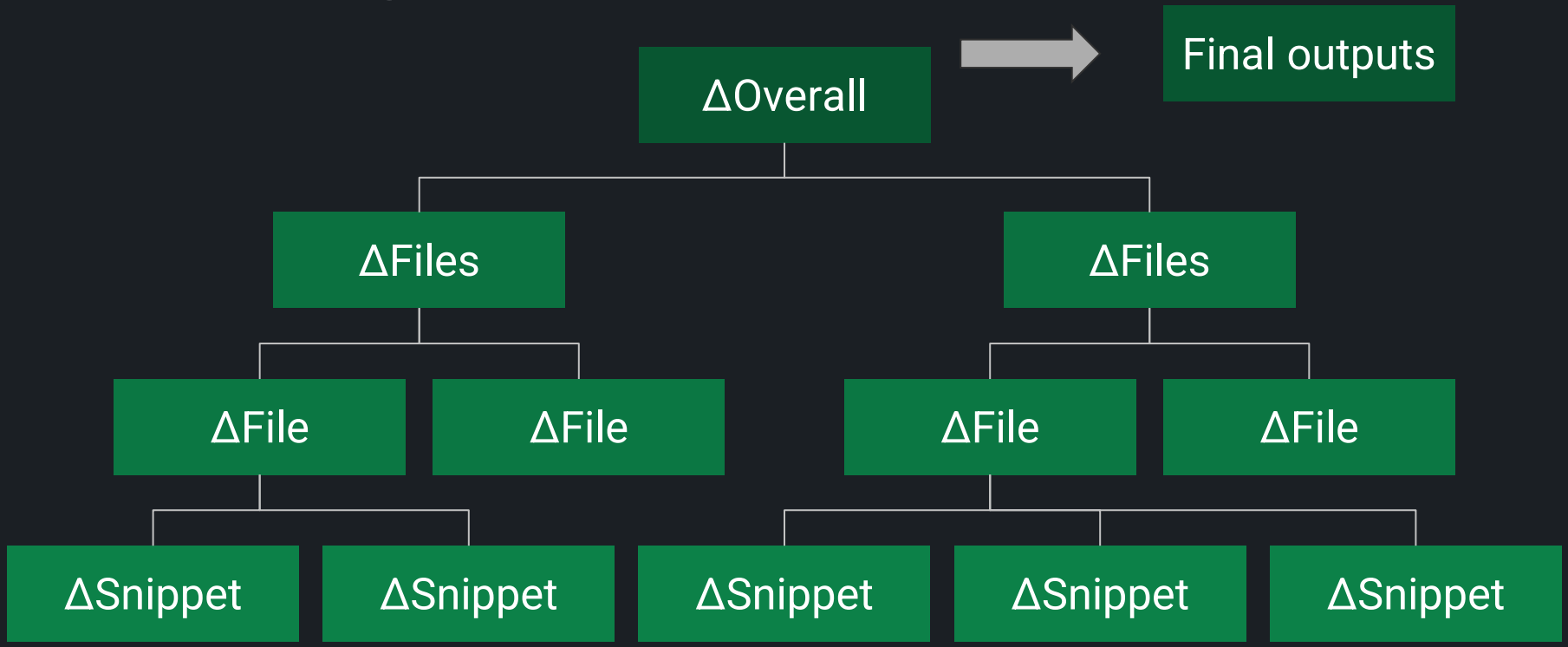
- Abstracts, keywords, descriptions
- Table of contents, documentation, walkthroughs

Use cases:

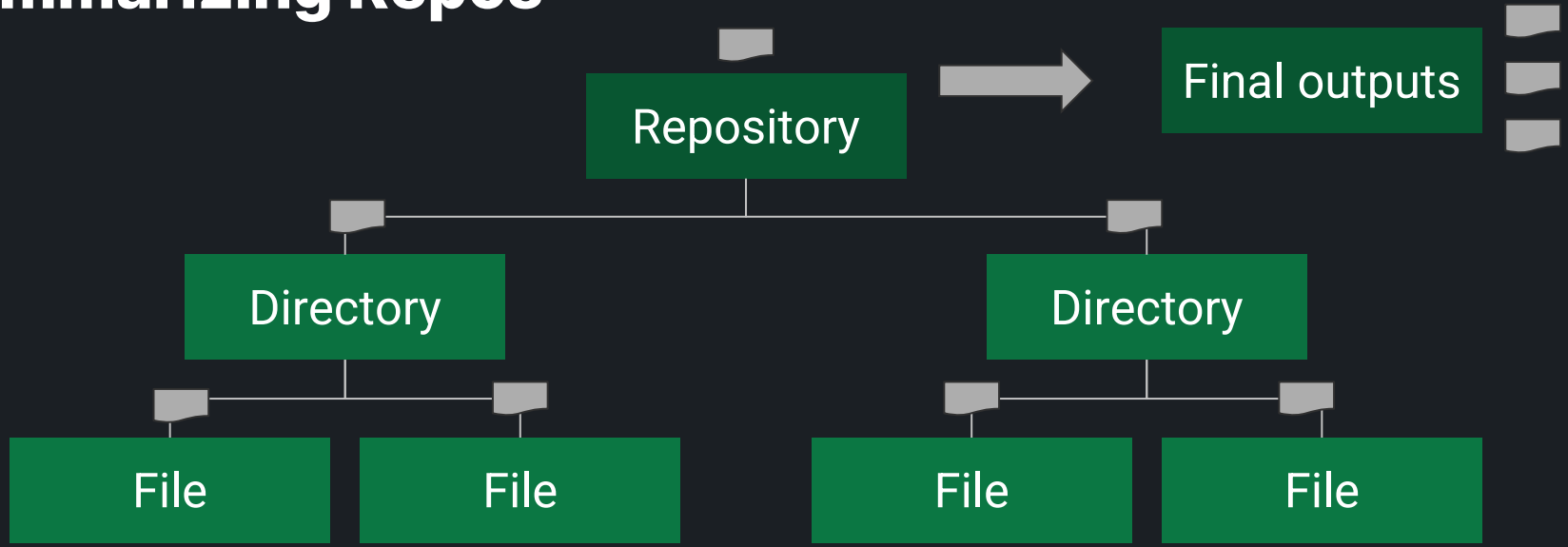
- Tool-tips
  - Infopanel content
- Human-authoring
- Search

Esp. when **unfamiliar** with content, **pressed for time**, or **small-factor devices**

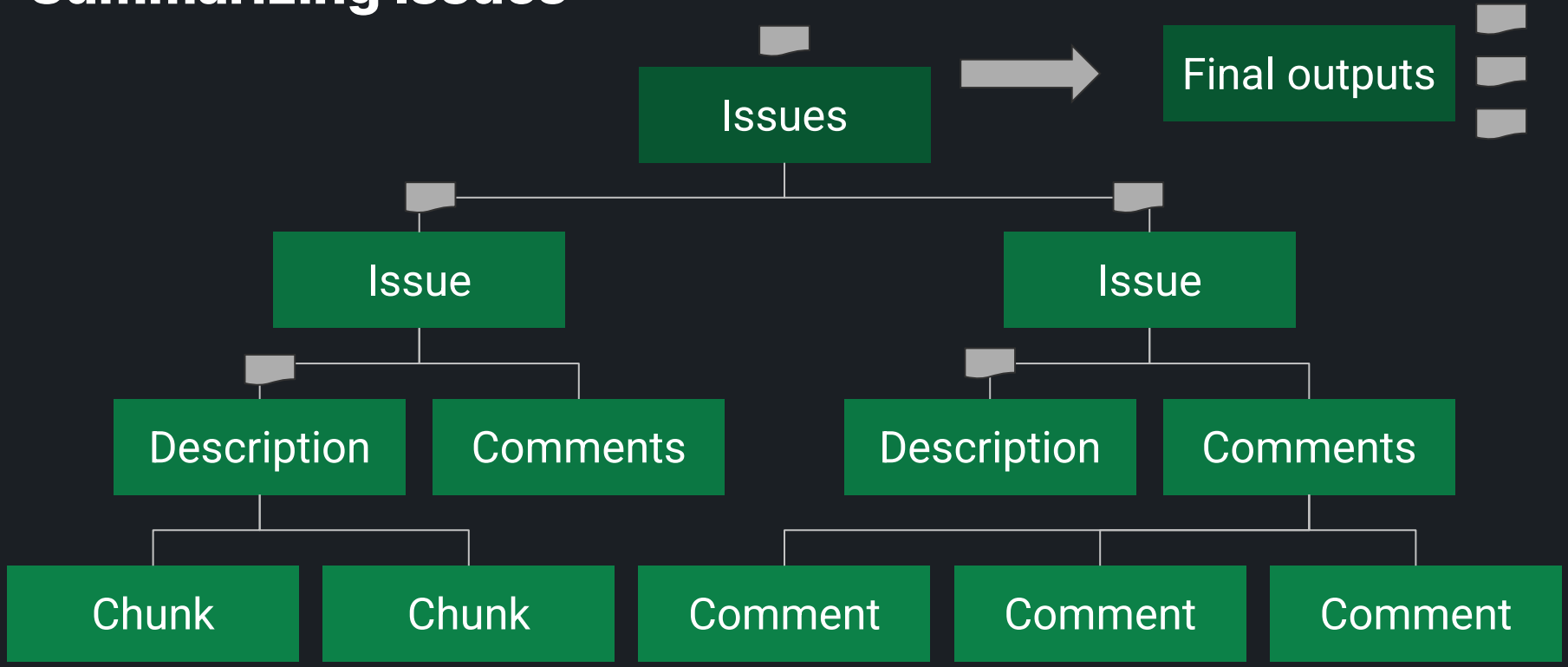
Summarizing PRs



Summarizing Repos



Summarizing Issues



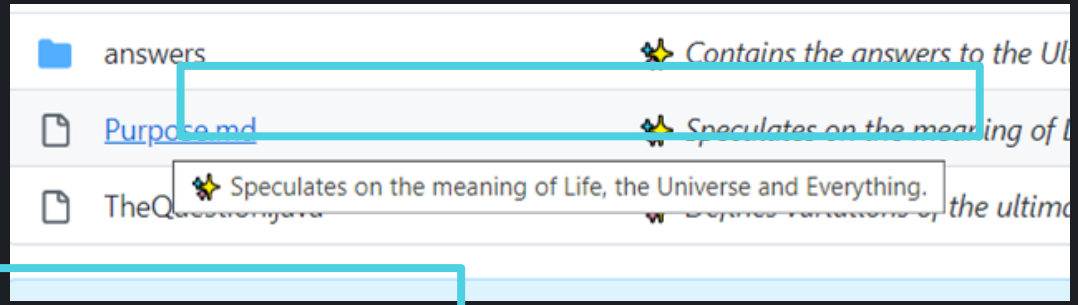
What to do with outputs?

- Ideally all managed by a “**summarization service**”
  - Akin to a “search service” (managing indexes and queries)
  - Akin to “thumbnail service” (managing thumbs across client/server)
  - This should be owned by GitHub, and aligns with our mission
- Faked in prototype
  - push summaries to a github branch in .thumbs directories
  - Stable naming convention for branch and thumb files
  - Thumbs can be picked up
    - by prototype tooling
    - flowed into other tooling

Examples

- code-retrieval
- HeyGitHub
- prosehub
- prbot
- docpilot
- copilot
- testpilot
- ghost-text-prototype
- incremental-codeql
- fsharp
- error-prone
- github/github

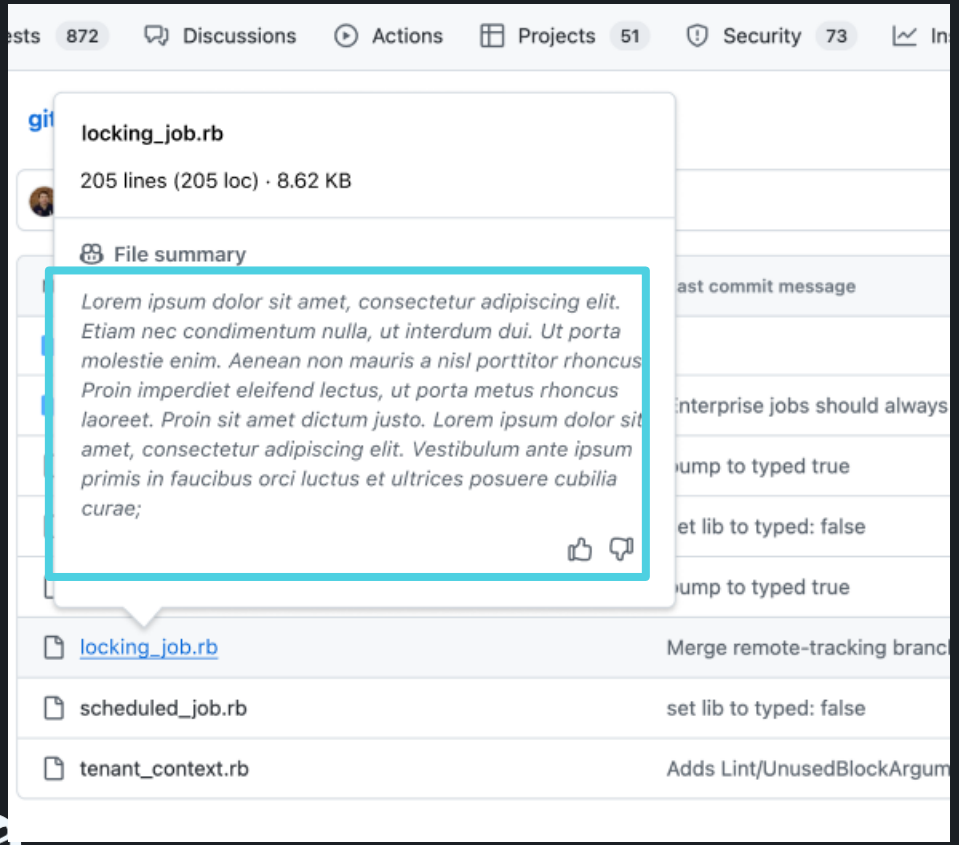
Surfacing in the UX (monolith spike)

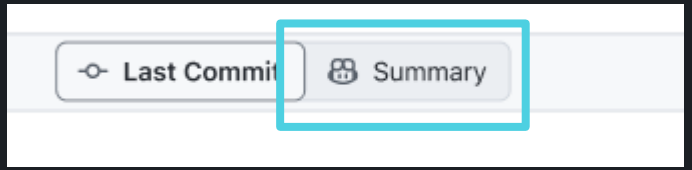


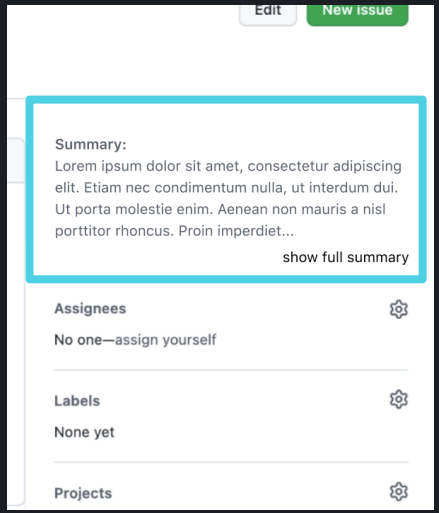
scorer	*Scripts and data for evaluating prbot summaries and walkthroughs.*	2 weeks ago
src	*Source code for prbot, a GitHub bot using OpenAI models.*	3 hours ago
test	*Tests, baselines, and scores for the prbot tool.*	4 hours ago
.dockerignore	*Files and directories to exclude from Docker build context.*	6 months ago
.env.example	*Environment variables for prbot, including credentials and proxy URL.*	4 months ago
.eslintignore	*Files and directories to exclude from ESLint.*	4 months ago
.eslintrc	*ESLint configuration for TypeScript files with plugins and rules.*	4 months ago

SPIKE: https://github.com/github/github/compare/users/dsyme/thumb-hacks

Surfacing in the UX (EPD prototypes)







[Full Figma Dump]

Sample prompts

- CoPilot for PRs:
  - ΔSnippet(s) → summary
  - ΔRepository → walkthrough
- Repo summarization:
  - Snippet/File → summary
  - Files → summary for directory

```
Let's write an overview of file [`app/api/admin/keys.rb`](F11H6919) in repository

## Snippet S11H3963 in `app/api/admin/keys.rb`:
```

Sample prompts

```
Write a very succinct, non-repetitive overview of directory [`app/api/admin/`](D0H4979).
* Only consider the content above.
* Forget everything else you know about this directory.
* Put the most important things first.
* IDs such as R0H9328 are repo trackers.
* IDs such as D3H8213 are directory trackers.
* IDs such as F2H3365 are file trackers.
* IDs such as S3H4298 are snippet trackers.
* Mention all file names and sub-directories immediately within the directory.
* Group together related files where possible.
* Enclose all code identifiers in single backticks.
* Each line you write should contain at least one relevant tracker.
* Put a single relevant repo, directory, file or snippet tracker at the end of each line using (tracker).
* Use only the trackers D0H4979,F0H00000,F11H6919,F12H9724,F13H2366,F14H8445,F18H1041, do not make up new trackers.
* Add markdown links to major external sites if they are certain to exist. Do not guess links.
* Enclose directory and file names in single backticks.
* Do not describe imported things such as functions, classes or types.
```

Sample prompts

```
* Where possible link directory and file names to a known tracker using markdown such as [`somedirectory`](D3H8213) or
[`somedirectory/somefile.ts`](F2H3365).
* Do not add further nested bullets. Start each line with at most two spaces.
* Use the following format:

START
Description of directory purpose, 300 words or less. (tracker)

* Title (tracker)
  * Short description, 100 words or less. (tracker)
  * Short description, 100 words or less. (tracker)
  ...
* Title (tracker)
  * Short description, 100 words or less. (tracker)
  * Short description, 100 words or less. (tracker)
  ...
```

Hallucination Reduction

- Label all inputs with “locators” for evidence
  - Requires model to “ground” its text
  - Throw away summary lines without evidence (see filterForHallucinatedLocators)
  - Not random, but not predictable by model
- Small solitary changes hallucinate the most
  - Pack changes together
  - Small descriptions for small changes
  - Require locators (throw away excess)
- Hallucinating “context” is a problem
  - e.g. “import X, export X as Y” - what is X??
  - Some retrieval would solve this

Hallucination Estimation

- Needs manual SME assessment
- “Hallucination Hunter” game is best I could think of
  - Summarize our own code
  - Uses our own team as source of SMEs
  - Winner will be announced at next in-person offsite!
  - Approx 1 minor hallucination/5min of SME analysis

Generic Summarization

```
export type Thing =
  | Repo
  | RepoTree
  | File
  | Directory
  | Snippet
  | Issue
  | IssueDescription
  | IssueComment
  | IssueCollection
  | Discussion
  | DiscussionDescription
  | DiscussionCommentAndReplies
  | DiscussionComment
  | DiscussionCommentReply
  | DiscussionCollection;
```

```
export type Directory = {
  kind: "directory";
  leaf: false;
  owner: string;
  repo: string;
  partial: boolean;
  directoryIndex: number;
  directoryName: string;
  sha: string;
  subThings: Thing[];
};
```

See summarizeThing

Generic Summarization

- Handful of operations for a new thing

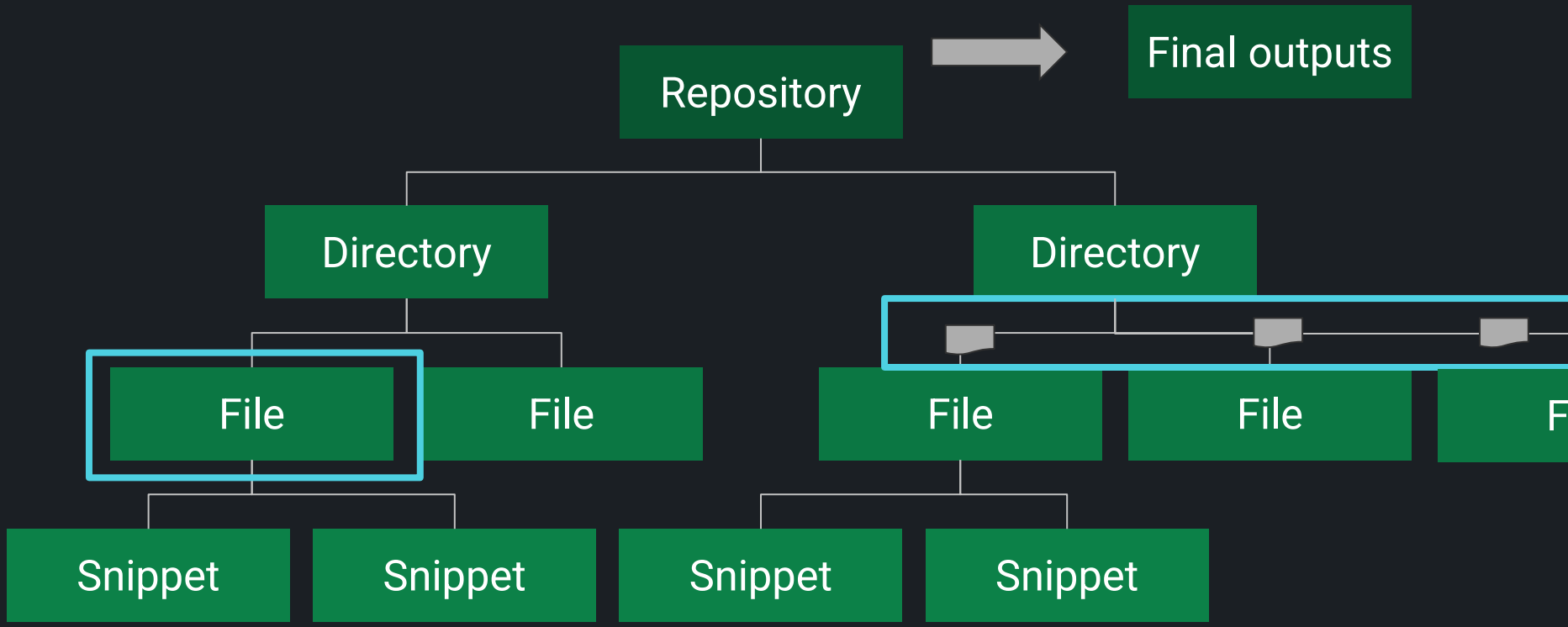
```
export function shortNameOfThing(thing: Thing): string {
  switch (thing.kind) {
    case "repo tree":
    case "repo":
      return `${thing.owner}/${thing.repo}`;
    case "directory":
      return thing.directoryName;
    case "file":
      return thing.fileName;
```

```
export function locatorForThing(thing: Thing) {
  switch (thing.kind) {
    case "directory":
      return locatorForDirectory(thing);
    case "file":
      return locatorForFile(thing);
    case "snippet":
      return locatorForSnippet(thing);
    case "issue description":
      return locatorForIssueDescription(thing);
    case "issue comment":
      return locatorForIssueComment(thing);
```

```
export function linkedNameOfThing(thing: Thing): string {
  const locator = locatorForThing(thing);
  switch (thing.kind) {
    case "repo tree":
    case "repo":
      return `repository [\`${thing.owner}/${thing.repo}\`](${locator})`;
    case "directory":
      return `directory [\`${thing.directoryName}\`](${locator})`;
    case "file":
      return `file [\`${thing.fileName}\`](${locator})`;
    case "issues":
```

See summarizeThing

Divide and conquer



See divideAndConquer

Flavours of output

- Locators:
  - Unfiltered
  - Markdown with locators (F16L2931)
  - Markdown with links to original
  - Markdown with links to to other summaries
- Length
  - Full → whatever // produced first
  - Paragraph → info panel
  - Sentence → hover
  - Phrase → listing

See shortenSummary

Incremental: Does code change?

Study: github/github “app” directory

Assume file granularity: “any change in file” ⇒ “resummarize file”

Size: 96MB of files

Change rates:

- **~2% change/day**
- **~10% change/week**
- **~25% change/month**

On a weekly basis:

- Week 14/2/2023 - 7.9MB
- …
- Week 30/11/2022 - 13MB

On a monthly basis:

- Month starting 21/1/2023 - 38 MB
- …
- Month starting 21/09/2022 - 23.9MB

= **Incremental monthly refresh is roughly 2x cost/year of one off**.

- Remediation : various approximations/skips mentioned above, or if it's done on-demand.

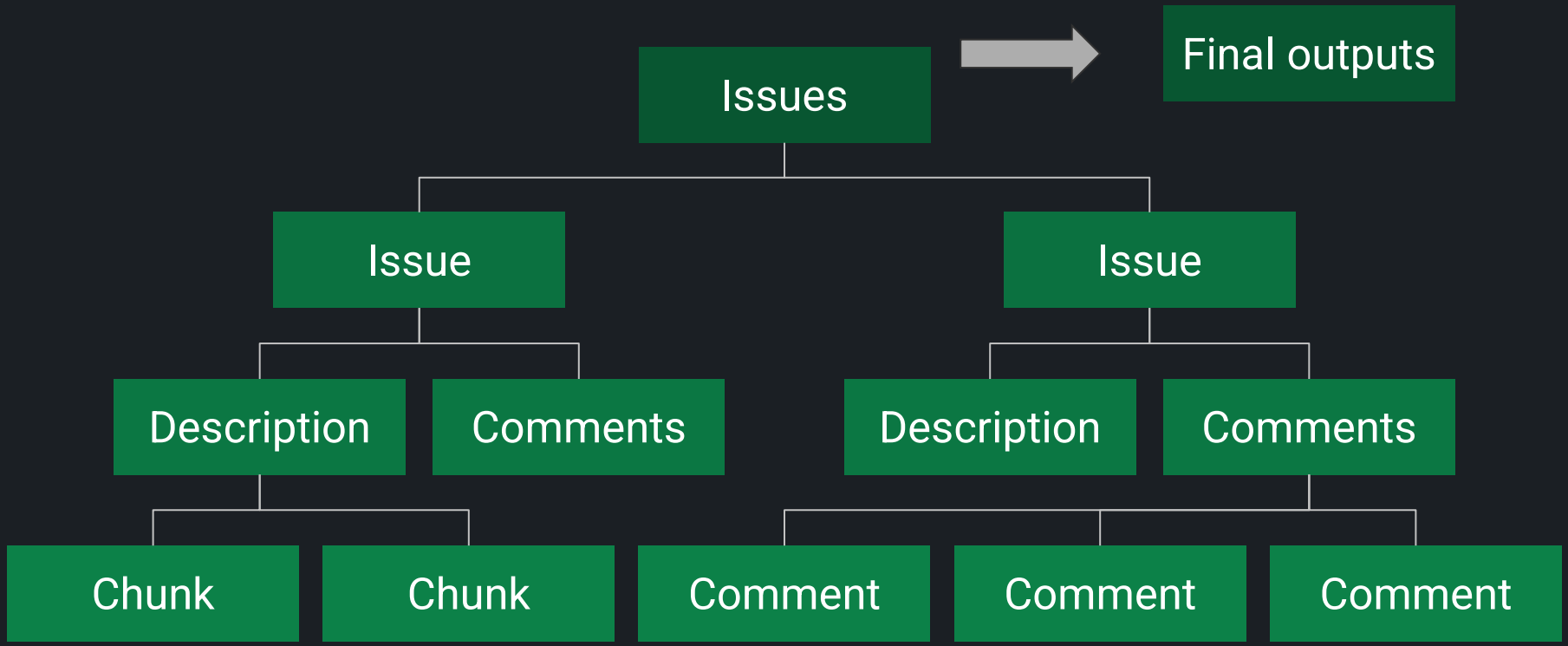
Possible futures

- The adjacent “product” is

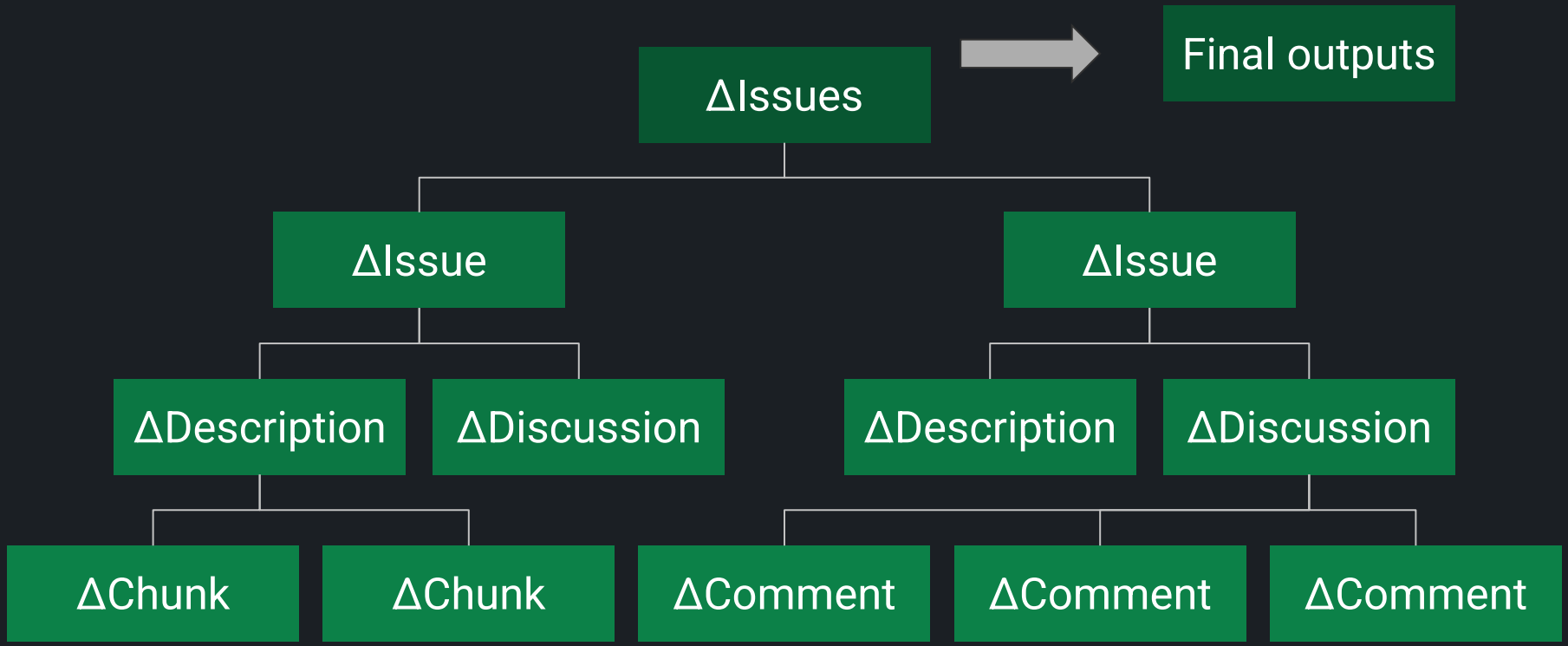
  pervasive, automatic, fresh summarization throughout github.com
  (of snippets, PRs, file, directory, repo, issue, discussions, projects, ….)

  - It’s a lot of engineering and compute
  - Fear implementation will likely end up scattered and duplicated
- Possible variations:
  - Summarization as a service (same content to VS, VSCode)
  - Focus on historical PRs and repos for enterprises
  - Focus on change - “what’s happened in last week”
  - Focus on specific pivots (e.g. dependencies, or UX, or …)

“Summarize all issues in this repo”



“Summarize issue discussions in this repo over last week”



A bit of fun - AI Stories for Repos

Based on final summary of all 1M LOC code in github/github “app” directory

SAGAN: ***The `app/` directory is a microcosm of the human condition, a mirror of our strengths and weaknesses, our hopes and fears, our dreams and realities.*** *It is a reminder of how far we have come, and how much further we can go....*

ATTENBOROUGH: ***In the vast and diverse ecosystem of GitHub, there is a hidden world of code that powers the web application millions of users rely on every day.*** *This world is the app directory, where we can find files and subdirectories for various features and services that GitHub offers.* ***Let's take a closer look at some of the inhabitants of this directory, and see how they interact and evolve.***

MACHIAVELLI: ***To the magnificent and generous prince,*** *who has graciously bestowed upon me the honor of examining the inner workings of his most esteemed and powerful enterprise,* ***I dedicate this humble treatise on the directory `app/` in the repository `github/github`****. ...the gateway to the prince's vast dominion of code and collaboration****. ..***

Thank you!

dsyme@github.com

Hierarchical summaries of entire repositories

Overall summary of 1M+ LOC code in github/github “app” directory.

Every chunk, file, directory, issue, discussion gets a summary too.

The directory `app/` contains files and subdirectories for the GitHub web application, which allows users to create, manage, and collaborate on software projects.

- `app/api/`
  - The GitHub API, which provides programmatic access to GitHub features and data.
  - Includes core API files, serializers, and subdirectories for various features such as organization, repository, code scanning, team, search, runtime, staff tools, enterprise, codespaces, deployments, and environment.
- `app/components/`
  - View components for various features and pages of the GitHub web application.
  - Uses the view_component library and the sorbet type checker.
  - Includes components for account verifications, GitHub Actions, GitHub Advanced Security, security advisories, animation, apps, beta features, billing, blobs, branch, businesses, checks, closables, code navigation, code scanning, codespaces, command palette, comments, commits, compare, compliance, conditional access, conduit, config as code, context switcher, copilot, dashboard, dependabot, dependency graph and review, devtools, diffs, discussions, environments, events, explore, feed, feed posts, files, fragments, flipper features, gists, and hovercards.
- `app/controllers/`

Determining Hallucination Rates

- This is hard
- Subject Matter Experts must assess
- Ran a “Hallucination Hunter” game internally to incentive experts to find hallucinations in descriptions of their own code
  - Very few in final summaries

RLA

- RLA = **Required Logical Accuracy**
- Same outputs have different RLA for different tasks!
  - Code Review = very high RLA
  - “A Quick Overview” for the Newcomer = medium RLA
  - Poem = low RLA
- UX really matters, can reduce RLA

Summarize all the things

Code ⇒ Summary (“Explain this code” in Copilot VS Code)

PR ⇒ Summary (Copilot for PRs)

File ⇒ Summary

Directory ⇒ Summary

Issue ⇒ Summary

Discussion ⇒ Summary

Repo => Summary

(multiple AI calls needed == hierarchical summarization)